SE4041 Assignment 02 Report



Student ID – IT23404564

Student Name – H.M.H.Lakshan

SnapSlip ios application and SnapSlip Spatial application

Part A GitHub Repository : https://github.com/it23404564-sliit/SnapSlip-ios-app/tree/main/SnapSlip

Part B Github Repository : https://github.com/it23404564-sliit/SnapSlip-ios-app/tree/main/SnapSlipSpatial

Introduction

This report documents two connected Apple-platform apps. **SnapSlip** (Part A) is an iPhone and iPad app that turns a photo of a paper receipt into a categorised, budget-aware expense, entirely on the device. **SnapSlip Spatial** (Part B) is a visionOS prototype that turns the same spending data into a 3D city and an immersive "budget orbit" on Apple Vision Pro.

- **Problem:** in Sri Lanka most everyday purchases end with a paper slip, and budgeting apps fail because typing every bill is tedious and charts rarely change behaviour.
- **Part A solution:** VisionKit captures the receipt, Vision reads it, a custom parser extracts the shop, date, items and total, and a four-layer on-device categoriser (personal rules, a Core ML text classifier, local keyword rules and Natural Language embeddings) suggests the category and learns from every correction. Data is stored with SwiftData; dashboards use Swift Charts and custom animated components.
- **Part B solution:** a visionOS app with three scene types -a window, a volumetric "Spending City" and a mixed-immersion "Budget Orbit" -built with RealityKit, a custom Entity-Component-System, and gaze-and-pinch interaction. It imports the CSV exported by the iPhone app.
- **Scale:** the iOS app has 39 Swift source files across ten screens; the visionOS app has 10 source files.There are 68 automated tests: 46 unit tests and 10 UI and performance tests for iOS, and 12 unit tests for visionOS.
- **Privacy:** neither app makes any network call. Receipts, recognised text and the machine-learning model never leave the device.

Results that depend on running the apps -test outcomes, model accuracy, performance measurements and user acceptance testing -are recorded in sections 6, 7 and 11.6; their tables are left for the measured values.

Part A -SnapSlip for iPhone and iPad

1. Introduction

SnapSlip turns a photo of a paper receipt into a categorised, budget-aware expense in a few seconds, with all processing on the device. SnapSlip Spatial, the visionOS companion, lets the user stand inside their month of spending.

The problem. Most everyday purchases in Sri Lanka still produce a paper bill: supermarkets, bakeries, pharmacies, fuel sheds and tuk rides. Budgeting apps expect people to type each one in, so most people give up within weeks. Bank-linked trackers miss cash entirely, and many students do not have cards at all.

The solution. SnapSlip makes capture the cheapest step:

1. Scan the receipt with the document camera, or pick a photo.
2. Vision reads the text, and a parser extracts the shop, date, line items, total and currency.
3. A layered on-device model suggests the category and shows how confident it is and which engine decided.
4. The user confirms or corrects it. Corrections are remembered for that shop next time.
5. Dashboards show budget used, a month-end projection, category breakdowns and trends.

Scope against the brief.

Brief requirement	Where SnapSlip meets it
Three or more navigable screens	Overview, Expenses, Expense detail, Scan flow, Insights, Category detail, Settings, Category budgets, Learned shops, Onboarding
Advanced UI elements	Custom animatable ring Shape, matched-geometry chips, zoom transitions, gesture-driven receipt viewer, staggered card animations, laser scanning state
SwiftUI customisation	Custom ViewModifiers (card, shimmer, appear), two ButtonStyles, a custom Layout (FlowLayout), user-selectable accent themes
NavigationStack	One typed NavigationStack per tab, driven by a central router; NavigationSplitView on iPad
Data persistence	SwiftData with a versioned schema, four models, a cascade relationship, external image storage
Emerging technology	Vision OCR, VisionKit document camera, Core ML text classifier, NaturalLanguage embeddings
Accessibility	Dark Mode colour assets, Dynamic Type adaptive rows, VoiceOver labels, Reduce Motion
Testing	Swift Testing unit tests, XCUITest UI tests, launch and scroll performance tests, UAT plan

2. Target audience

The primary audience is Sri Lankan university students and early-career professionals aged 18–30 who live on a fixed monthly allowance or salary and pay mostly in cash. They own an iPhone, are comfortable with the camera, and care about privacy.

Persona	Context	Need	How SnapSlip answers it
Nimal, 21, undergraduate in Malabe	Rs. 40,000 monthly allowance; canteen, bus, tuk, stationery	Know mid-month whether the allowance will last	Budget ring, daily average and month-end projection on the first screen
Tharushi, 26, software engineer in Colombo	Shares groceries with flatmates; pays cash and card	Track shared supermarket bills without typing items	Line items read from the receipt; CSV export to share with flatmates
Dilan, 29, sales executive	Claims fuel and client lunches back	Keep photos of receipts with the right category	Receipt image stored with the expense, zoomable, searchable by shop

Why these users, and how the design follows from them:

- **Cash and paper first.** Capture starts from the receipt, not a bank feed. The parser is tuned for day-first dates, Rs./LKR and thousands separators.
- **Local merchants.** Keyword rules and training data cover Keells, Cargills, Arpico, PickMe, Ceypetco, Dialog, Osu Sala and others. Unknown shops fall back to language embeddings.
- **Privacy.** Spending is personal. Nothing is uploaded: OCR, ML and storage all run on the device, which the app says during onboarding and in Settings.
- **Low effort.** One tap opens the scanner from any main screen. Review fields are pre-filled, and a correction teaches the app.
- **Inclusive.** Dynamic Type up to accessibility sizes, VoiceOver, Dark Mode and Reduce Motion are supported throughout, because the audience includes people with visual or motion sensitivity.

3. Features and app functionality

SnapSlip has eleven screens reached through four tabs, two sheets and a full-screen scan flow. Every feature works offline and stores its data with SwiftData.

Screen	What the user can do	Key implementation
Onboarding	Swipe three intro pages; set name, currency and monthly budget	Paged `TabView`, animated SF Symbols, `@AppStorage`
Overview	See budget used, today's spend, daily average, month-end projection, category chart, recent expenses	`BudgetRingView`, Swift Charts, two narrow `@Query` fetches
Expenses	Search by shop or note, filter by category, swipe to edit or delete, long-press preview, duplicate	`#Predicate` filtering in the store, day grouping, context menu

Screen	What the user can do	Key implementation
Expense detail	Read amount, shop, date, category confidence, items, note and recognised text; zoom the receipt; share; edit; delete	Zoom transition, `ZoomableImageView`, `ShareLink`
Add / edit expense	Enter or change every field, including line items with quantities	`ExpenseDraft` value copy, live validation
Scan flow	Choose camera, photo, sample receipt or manual entry; watch processing; review and correct; save	`ScanViewModel` state machine, Vision bounding-box overlay
Insights	Switch range (1, 3, 6 months); explore the donut; compare pace with budget; see trend and top places	`chartAngleSelection`, `RuleMark`, `AreaMark`
Category detail	See a category's month total, budget progress and all its expenses	Store-level category predicate
Settings	Change budget, currency, theme and appearance; export CSV; load sample data; erase everything	`ShareLink` with a `Transferable` CSV
Category budgets	Set or clear a monthly limit per category	`ExpenseRepository.setBudget`
Learned shops	Review, change or forget categories learned from corrections	`MerchantRule` model

Robustness built into the features:

- **Validation.** Save stays disabled until a shop name and an amount between 0 and 100,000,000 are entered; dates more than a day in the future cannot be chosen.
- **Safe editing.** Forms edit a copy of the data, so Cancel never leaves half-saved changes. Destructive actions (delete, erase all) ask for confirmation.
- **Failure paths.** If Vision finds no text or a photo can't be opened, the scan flow shows the reason with *Try again* and *Enter manually*. Every list has an empty state that explains what to do next.
- **Data integrity.** Deleting an expense also deletes its line items and receipt image; replacing items leaves no orphaned rows (covered by unit tests).

Functional verification checklist :

- Onboarding completes and the settings are kept after relaunch
- Each sample receipt scans with the correct shop and total
- A camera scan works on a real iPhone
- Correcting a category is remembered on the next scan from the same shop
- Search, category filter, swipe actions and context menu work
- Budgets and category budgets update the Overview colours
- CSV export opens in Files or Numbers
- The app works on iPad in portrait and landscape

4. UI/UX design, visual design and branding

The visual language is calm and card-based, with one strong accent and category colours that carry meaning. Every animation explains a state change rather than decorating it.

Visual design and branding

- **Brand.** The name SnapSlip pairs the action (snap) with the local word for a receipt (slip). The app icon is a white receipt with a torn edge inside viewfinder corners on a teal gradient. It has light, dark and tinted variants for iOS 18 home-screen styles.
- **Colour.** The default "Lagoon" teal is the accent. Users can switch to Sunset, Tea Estate or Orchid in Settings, and the whole app re-tints live through an environment value. Each of the nine categories has a fixed mid-tone colour that stays readable on light and dark cards.
- **Surfaces.** Background and card colours live in the asset catalogue with separate dark appearances. Cards have a soft shadow in light mode and a hairline border in dark mode, where shadows disappear.
- **Type.** Only semantic text styles are used, so Dynamic Type scales everything. Amounts use monospaced digits so columns stay aligned and the rolling-number transition doesn't jitter.



Figure 4.1 -The SnapSlip app icon (1024 × 1024). Dark and tinted variants ship in the same asset catalogue.

Interaction and animation

Element	What the user sees	Why
Budget ring (custom animatable `Shape`)	Fills with a spring on appear; a second red ring laps it when over budget	Overspending must be visible at a glance
Amount text	Digits roll to the new value	Shows that the total changed, not just that the screen redrew
Scanning state	A light sweeps over the receipt while Vision runs	Turns a 1–2 s wait into visible progress
Text boxes	Vision's detected regions pop in one after another over the receipt	Builds trust by showing what the app read
Category and filter chips	The selected background glides between chips (matched geometry)	Makes the choice feel direct and reversible
Expense rows → detail	iOS 18 zoom transition from the row; standard push on iOS 17	Keeps context when drilling in
Receipt viewer	Pinch to zoom, drag to pan, double-tap to toggle, drag down to dismiss	Matches the Photos app, so no instructions are needed
Floating scan button	A soft pulse and a springy press	The main action is always one tap away

Haptics confirm saves, deletes and selections (`sensoryFeedback`). All custom animations check **Reduce Motion** and fall back to simple fades.

Adaptive layout

- iPhone uses a `TabView` with a `NavigationStack` per tab. iPad and wide landscape windows use a `NavigationSplitView` with a sidebar and a persistent scan button.
- Dashboard and Insights cards switch from one column to two in regular width.
- The scan review shows the receipt beside the form on wide screens and above it on phones.
- Expense rows switch from a horizontal to a vertical layout at accessibility text sizes (`AnyLayout`). Stat tiles use `ViewThatFits` to stack when space runs out.
- Category chips use a custom `FlowLayout` that wraps for any width, orientation or text size.

Accessibility

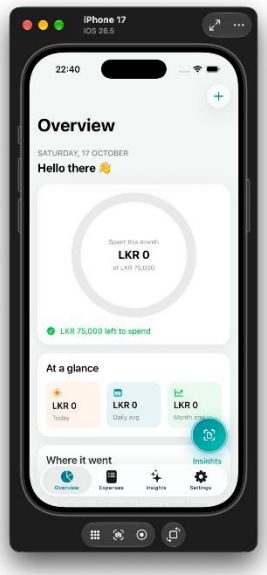
- Every control has a label. Charts and the budget ring expose values ("68 percent"), and composite rows are combined into a single VoiceOver element.
- Selected chips and themes carry the *selected* trait.
- The receipt viewer offers a named *Close* action for VoiceOver users, who can't perform the drag-down gesture.

- Colour is never the only signal: budget status also has an icon and a sentence ("Careful -only Rs 9,000 left").

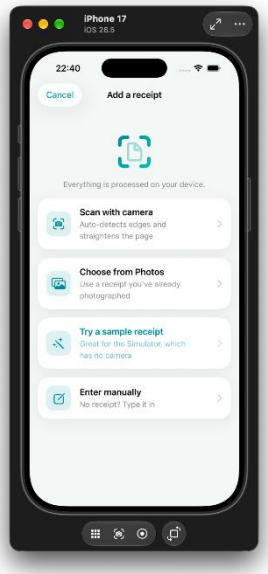
Screens

The figures below show the screens on the devices they were designed for.

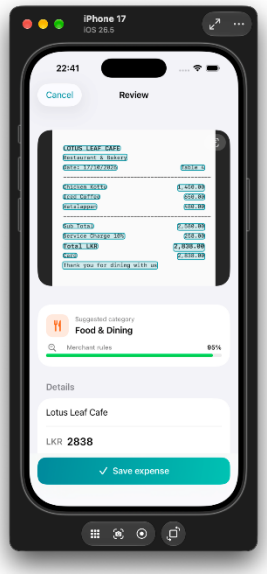
Screen	Device and setting	What it should show
Onboarding	iPhone 16, light	The "Smart, private categories" page
Overview	iPhone 16, light and dark	Budget ring part-filled, stat tiles, category chart
Scan -processing	iPhone 16	The light sweep over the sample supermarket receipt
Scan -review	iPhone 16	Text boxes on the receipt, suggested category and confidence bar
Expenses	iPhone 16	A category filter selected and a swipe action open
Insights	iPhone 16, dark	Donut with one category selected, pace chart
Overview	iPad Pro 13-inch, landscape	Sidebar navigation and two-column cards
Expenses	iPhone 16, largest accessibility text size	Rows switched to the vertical layout



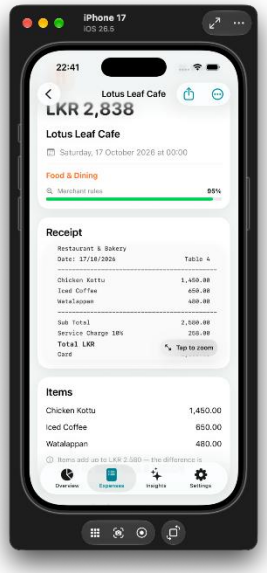
Overview screen



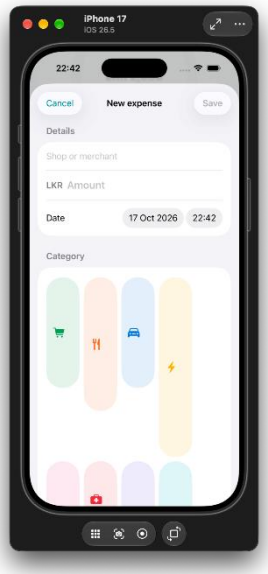
Add receipt screen



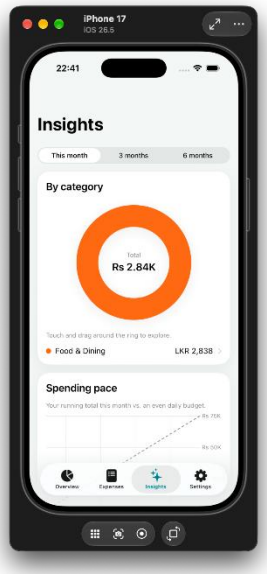
Scan -processing screen



Scan -review screen



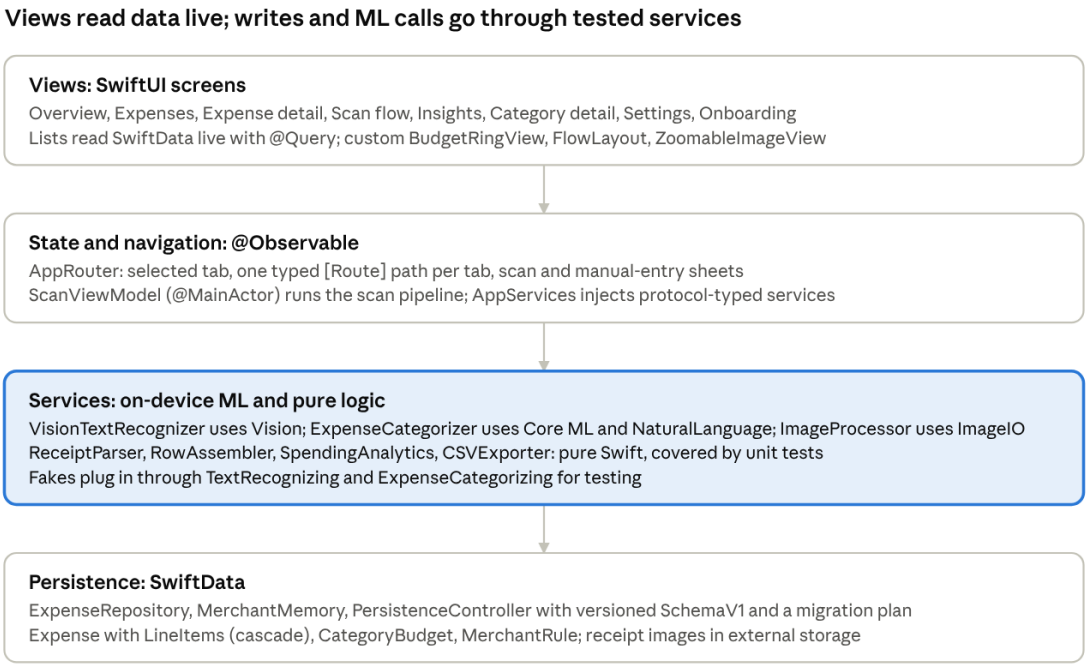
Expenses screen



Insights Screen

5. Architecture, code quality and technical documentation

SnapSlip is a SwiftUI app with an MVVM-style layering: views observe SwiftData directly for lists, while writes, OCR and ML go through small protocol-typed services that are injected and unit-tested. It targets iOS 17+, uses Swift 5 language mode and has no third-party dependencies.



SnapSlip architecture · 4 layers

Each layer only calls the one below it, apart from @Query reads, which keep lists live without a view model.

Modules

Folder	Key types	Responsibility
App	SnapSlipApp, AppRouter, AppServices, AppSettings	Builds the container, injects services and router, holds navigation state and preference keys
Models	Expense, LineItem, CategoryBudget, MerchantRule, ExpenseDraft, ExpenseCategory	SwiftData models plus a value-type draft for forms
Persistence	PersistenceController, ExpenseRepository, MerchantMemory, SampleData	Versioned schema, CRUD, personalisation rules, deterministic demo data
Services	VisionTextRecognizer, RowAssembler, ReceiptParser, ExpenseCategorizer, ImageProcessor, SpendingAnalytics, CSVExporter	OCR, parsing, categorisation, image downsampling, analytics, export
DesignSystem	Theme, CardModifier, ShimmerModifier, PressableButtonStyle, BudgetRingView, FlowLayout, ZoomableImageView	Tokens and reusable custom components

Folder	Key types	Responsibility
Views	Root, Onboarding, Dashboard, Expenses, Scan, Insights, Settings	Screens; ScanViewModel drives the scan pipeline

Navigation

AppRouter is an @Observable object in the environment. It holds the selected tab, a typed [Route] path for each tab, and the scan and manual-entry sheets. Route is a Hashable enum (expenseDetail, category, categoryBudgets, learnedMerchants), and every destination is registered once in TabRootContent. Programmatic navigation is one call, for example router.push(.expenseDetail(expense), on: .expenses) after a scan is saved. Each tab keeps its own back stack when switching tabs.

Data model and persistence

Model	Main fields	Notes
Expense	id (unique), merchant, amount, currencyCode, date, categoryRawValue, note, recognizedText, categoryConfidence, categorySourceRawValue, isScanned, createdAt	receiptImageData and thumbnailData use .externalStorage, so photos live as files next to the store
LineItem	name, price, quantity, position	Cascade-deleted with its expense (inverse relationship)
CategoryBudget	categoryRawValue (unique), monthlyLimit	Optional per-category limit
MerchantRule	merchantKey (unique), displayName, categoryRawValue, timesApplied, updatedAt	Learned from user corrections

- The schema is declared as SnapSlipSchemaV1 with a SchemaMigrationPlan, so a future V2 can ship a migration stage instead of wiping data.
- Lists use @Query with predicates pushed into the store. For example, the dashboard fetches only the current month, recent items use fetchLimit = 5, and search and category filters run as a single #Predicate.
- Forms edit an ExpenseDraft value copy, so Cancel never leaves half-edited data. ExpenseRepository applies drafts, replaces line items without leaving orphans, and manages budgets.
- Lightweight preferences (name, currency, monthly budget, theme, appearance) use @AppStorage.
- UI tests launch with -uiTesting, which switches to an in-memory store with fixed settings.

Concurrency and performance design

- Image decoding, OCR and parsing run off the main thread (Task.detached). The view model is @MainActor, so UI state changes stay on the main thread.
- ImageProcessor decodes camera photos straight to 2048 px for OCR, 1600 px for storage and 240 px for thumbnails using ImageIO. This avoids holding a 12–48 MP bitmap in memory.
- An NSCache keeps decoded thumbnails and frees them automatically under memory pressure.
- OCR is wrapped in an OSSignposter interval, so it appears in Instruments under Points of Interest.

Coding standards and documentation

- **Naming and structure** follow the Swift API Design Guidelines. Files are grouped by layer (App, Models, Persistence, Services, DesignSystem, Views), and most files hold one main type.
- **Comments.** Every file opens with a header that states its purpose. Non-obvious decisions are explained where they are made, for example why Vision observations are regrouped into rows, or why the detail screen dismisses before deleting.
- **Safety.** Optional values are unwrapped explicitly. The only force-tries are constant regular expressions, where a failure is a programming error found on first launch. The app stops with a clear message only if the database cannot be created.
- **Concurrency.** UI state is `@MainActor`; heavy work runs in detached tasks; value types passed between threads are `Sendable`.
- **Testability.** Services are reached through protocols (`TextRecognizing`, `ExpenseCategorizing`), and pure logic (parser, analytics, CSV) has no UI dependencies.
- **Localisation-ready.** User-facing text uses `LocalizedStringKey` and `String(localized:)`, so it can be exported to a String Catalog for Sinhala and Tamil.
- **No third-party code.** Only Apple frameworks are used, which removes licensing and supply-chain risk.

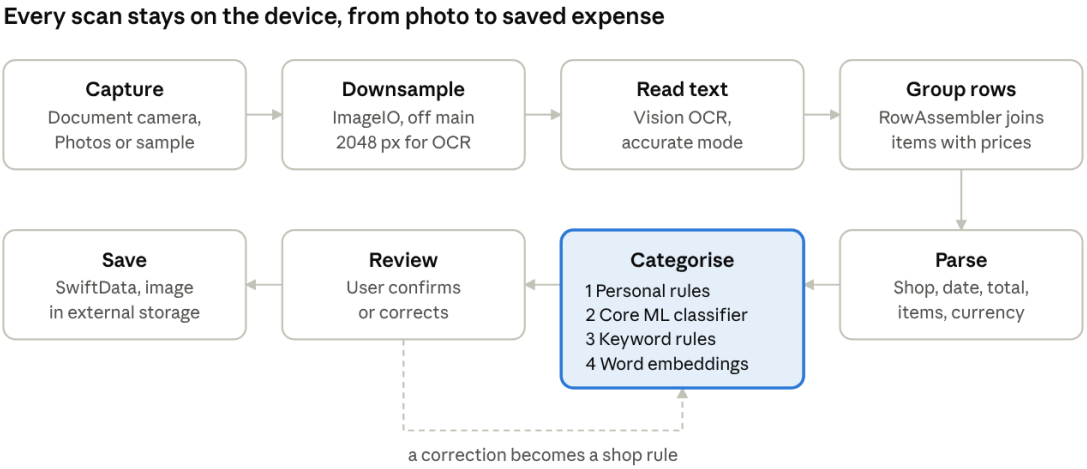
Version control

Each app has its own GitHub repository with a README (build, run and test instructions) and an `AI_USAGE.md` log. Work is organised as follows:

Branch	Purpose
`main`	Stable, demo-ready code; merged only through pull requests
`develop`	Integration branch for finished features
`feature/scan-flow`, `feature/ml-categoriser`, `feature/insights`, `feature/settings`, `feature/tests`	One branch per feature, merged into `develop` by pull request

6. Emerging technology: on-device vision and machine learning

The app's core feature, receipt to categorised expense, is built from four Apple machine-learning frameworks working in sequence. The ML is the feature, not an add-on.



scan pipeline · 8 steps, 4 categorisation layers

The first layer that is confident enough decides the category; the dashed loop is how SnapSlip learns from the user.

Capture: VisionKit

`VNDocumentCameraViewController` provides automatic edge detection, perspective correction and auto-capture on device. Photos from the library and four rendered sample receipts use the same pipeline, so the Simulator can demonstrate everything.

Reading: Vision OCR

`VNRecognizeTextRequest` runs with `.accurate` recognition and language correction. Vision often returns an item name and its price as two observations even though they sit on the same printed row. `RowAssembler` regroups observations whose vertical centres overlap by at least half a line height and joins them left to right. Without this step, line items and totals were frequently separated from their amounts.

Understanding: ReceiptParser

A pure-Swift parser turns rows into structured data:

- **Total**: searched in priority order (grand total, net total, total due, amount due … total), skipping subtotal, item-count and savings lines. If the amount is on the next line it is taken from there. The fallback is the largest decimal amount that isn't cash given or change.
- **Date**: day-first numeric formats, ISO dates and named months ("5 Sep 2026", "Sept 21, 2026"). Dates in the future or before 2000 are rejected as OCR noise.
- **Merchant**: the first of the top five lines that is mostly letters and isn't a header such as TAX INVOICE or WELCOME.
- **Line items**: rows ending in a price, with quantities ("2 x Bananas") and leading barcodes removed, and whole-word filtering of tax, payment and total rows.
- **Currency**: Rs./LKR, $, €, £, ₹.

Categorising: a layered model

Order	Layer	Technology	When it decides	Example
1	Personal rules	SwiftData `MerchantRule`	The user has corrected this shop before	"Green Cabin" → Food & Dining at 99%
2	Text classifier	Core ML model trained with Create ML, run through `NLModel`	Model bundled and confidence ≥ 45%	Unknown café with "kottu" and "latte" on the bill
3	Keyword rules	Curated Sri Lankan brand and keyword lexicon, merchant weighted ×2	Any known keyword matches	"Keells Super" → Groceries
4	Word embeddings	`NLEmbedding` cosine distance to category anchor words	Any word is close to an anchor (distance < 1.0)	"Paracetamol" → Health

The review screen shows the suggested category, a confidence bar, which engine decided, and up to two alternatives as one-tap chips. If the user picks something else, SnapSlip stores a rule for that shop, so the app becomes personal with use. Rules can be reviewed, changed or deleted under *Settings → Learned shops*.

Training the Core ML model

`MLTraining/receipt_categories.csv` holds 1,200 labelled examples across nine categories, built from Sri Lankan merchant names and typical receipt words. Text is normalised identically at training and run time: lowercase, accents folded, letters only, words of two or more characters. `TrainClassifier.swift` trains a maximum-entropy `MLTextClassifier` with an 80/20 split and prints training, validation and test accuracy plus the confusion matrix. The resulting model is a few tens of kilobytes. The app loads the compiled model if it is present and otherwise falls back to layers 3–4.

Training data. 1,200 rows: 140 for each of the eight main categories and 80 for Other. 20% of the rows are held back as a test set (seed 42), and Create ML takes a further automatic validation split from the training rows.

End-to-end check with the sample receipts. The four rendered receipts exercise the whole pipeline. The expected category comes from the design of the keyword and model layers; record what the app actually shows.

Why this integration is meaningful

- It removes the most tedious step, typing, from expense tracking.
- It works offline and keeps financial data private.
- It is transparent: the user sees the confidence and source, and can correct it.
- It improves with use through the personal layer.

7. Testing, debugging and performance optimisation

Testing covers four levels: unit tests for the logic, UI tests for the main flows, performance metrics, and a user acceptance test with real users and receipts. Run ⌘U in Xcode with code coverage on, then paste your actual results into the result columns below.

Automated tests

Suite	Framework	What it checks	Tests
Receipt parser	Swift Testing	Full receipt, total priority, subtotal exclusion, next-line totals, fallback, amounts (4 cases), dates (6 formats), rejected dates, merchant headers, name tidying, item filtering, currency (5), empty input	13 tests, 25 cases
Expense categoriser	Swift Testing	10 Sri Lankan merchants, body-text fallback, merchant weighting, unknown → Other, normalisation, model flag, merchant-memory keys	7 tests, 16 cases
Spending analytics	Swift Testing	Category totals, monthly totals with empty months, cumulative daily, projection, budget levels (4), percent change, headline insight, top merchants	8 tests, 11 cases
Persistence	Swift Testing, in-memory SwiftData	Add with line items, blank items dropped, cascade delete, update without orphans, date-range fetch, category budgets, merchant memory, sample data, erase all	9 tests
Scan pipeline and helpers	Swift Testing, fake OCR	Draft built from OCR rows, error state, personal rule wins, correction learns, manual entry, image pipeline sizes, row assembly, Vision-to-SwiftUI box conversion, CSV escaping	9 tests
UI flows	XCUITest	Tab navigation, open detail, manual add then search, Save disabled until valid, full sample-receipt scan, category filter, theme and appearance, onboarding	8 tests
Performance	XCTest metrics	App launch time; scrolling and deceleration of the expense list	2 tests

The scan view model receives its OCR engine through the `TextRecognizing` protocol, so tests inject a fake recogniser that returns canned rows. This makes the full pipeline testable without Vision or a camera.

Debugging approach

- **Previews with sample data.** Most screens have a SwiftUI preview backed by an in-memory store filled with the deterministic sample data, so layouts were checked in light, dark and large-text modes without running the app.
- **Repeatable runs.** The `-uiTesting` and `-seedSampleData` launch arguments start the app with a clean in-memory store and known data. The same data is used for screenshots and the demo.
- **Xcode tools.** Breakpoints and the view debugger for layout problems; the memory graph debugger to check for leaks after repeated scans; Instruments (Allocations, Time Profiler, Hangs) with the `RecognizeText` signpost to time OCR.

User acceptance testing

Plan: 5 participants from the target group (students and young professionals), 20 minutes each, on a real iPhone with their own receipts. Measure task success, time on task and one SUS questionnaire per person.

#	Task	Success criterion
1	Complete onboarding and set a monthly budget	Reaches Overview with the budget shown
2	Scan a supermarket receipt and save it	Correct total and merchant, or fixed in under 30 s
3	Correct a wrong category, then scan the same shop again	Second scan suggests the corrected category
4	Find last week's pharmacy expense	Found with search or filter in under 20 s
5	Answer "Which category did you spend most on this month?"	Correct answer from Insights
6	Switch to Dark Mode and the largest text size; repeat task 4	Task still completed, nothing clipped
7	Export expenses as CSV	File opened in Files or Numbers

Record the SUS score, success rate per task, and the top three issues found and how you fixed them.

Performance optimisation

Area	Technique	How to verify
Memory	ImageIO downsampling at decode time (2048 / 1600 / 240 px); images in `.externalStorage`; `NSCache` thumbnails	Instruments → Allocations: peak memory while scanning a 12 MP photo
Load time	Narrow `@Query` fetches (current month; recent with `fetchLimit`); no work in `App.init`	`testLaunchPerformance` metric; Instruments → App Launch

Area	Technique	How to verify
	beyond creating the container	
Responsiveness	OCR, decoding and parsing off the main thread; signposted OCR interval	Instruments → Points of Interest shows the `RecognizeText` interval; Hangs instrument shows none
Scrolling	Lightweight rows, cached thumbnails, store-level filtering	`testScrollingExpensesPerformance` (hitch ratio)
Battery	No network, no background work, OCR only on demand	Energy gauge in Xcode during a scan

The review screen shows "Processed on device in …" for every scan, which gives a simple end-to-end timing to quote in the presentation.

8. Development challenges

The hardest problems were turning noisy OCR text into reliable numbers and keeping SwiftData, navigation and animation in step. Each challenge below has the fix that went into the code; add any further ones you hit while building and testing in Xcode.

Challenge	What went wrong	Resolution
Items split from prices	Vision returned "Fresh Milk" and "450.00" as separate observations, so no row ended in a price	`RowAssembler` merges observations on the same printed row by vertical overlap, left to right
Picking the right total	Receipts print Sub Total, Total, Cash and Change, all with amounts	Keyword priority list, an exclusion list, and a fallback that ignores cash given and change
Day-first dates	"08/10/2026" means 8 October in Sri Lanka but 10 August in US-locale parsers	Fixed POSIX formatter with day-first formats; rejects future and pre-2000 dates; tests cover six formats
Over-eager keyword filters	"Ground Coffee" was dropped because it contains "round"; "Eggs (10)" lost its bracket	Whole-word regex filtering; trimming limited to separators, not all punctuation
Vision vs SwiftUI coordinates	Vision boxes are normalised with the origin bottom-left; SwiftUI is top-left in points	`RecognizedTextOverlay.convert` flips the y-axis; images are pre-rotated so EXIF orientation can't skew boxes; unit-tested
Memory spikes	Decoding a 12 MP photo for OCR and thumbnails used hundreds of MB	ImageIO thumbnails at decode time; external storage for images; `NSCache`
Orphaned line items	Replacing an expense's items left the old `LineItem` rows in the store	Delete old items explicitly before assigning new ones; test checks the row count
Deleting the model being	Deleting from the detail screen could	Dismiss first, then delete after the transition

Challenge	What went wrong	Resolution
shown	render a deleted object during the pop	
Live Save button	`TextField(value:format:)` only commits on Return, so Save stayed disabled	The amount field binds to text and parses on every keystroke
CSV dates shifting a day	`ISO8601DateFormatter` defaults to GMT; a 00:30 Colombo purchase was exported as the previous day	Formatter set to the user's time zone; test made time-zone independent
Testing without a camera	The Simulator has no camera, and Vision results vary	Rendered sample receipts; `-uiTesting` launch argument with an in-memory store; a fake recogniser for unit tests
iOS 17 and 18 support	Zoom navigation transitions exist only on iOS 18	`zoomSource` and `zoomDestination` modifiers use `#available` and fall back to the standard push

9. Innovation and creativity

SnapSlip's originality is in where it starts and how openly it uses machine learning: it begins from the paper receipt in your hand, and it shows its reasoning and learns from you.

Typical approach in budgeting apps	SnapSlip's approach	Why it matters
Link a bank account, or type each expense	Scan the paper receipt; everything is filled in	Works in a cash economy and for students without cards
Generic, US-centred parsing	Tuned for Sri Lanka: day-first dates, Rs./LKR, local shop names in the lexicon and training data	Fewer corrections for the actual audience
A category appears with no explanation	Confidence bar, the engine that decided, one-tap alternatives, and Vision's text boxes drawn on the receipt	Users can see why and trust or fix it
The same model for everyone	A personal rule layer runs before the model and is editable in *Learned shops*	Accuracy improves with each use, and the user stays in control
Receipts uploaded to a server for OCR	Zero network code; OCR and ML on the device	Financial data stays private
Phone-only charts	Export to SnapSlip Spatial and walk around the month on Vision Pro	A new way to understand spending (Part B)

Smaller creative touches support the same idea: four accent themes named after Sri Lankan scenes (Lagoon, Sunset, Tea Estate, Orchid), rendered sample receipts so the full pipeline can be shown in the Simulator, and a "processed on device in …" timer on every scan.

10. Deployment readiness and live demonstration

The app is structured for App Store release; the remaining steps are signing, a privacy manifest and a TestFlight build.

Item	Status
App icon with light, dark and tinted variants	Done
Display name, bundle ID `lk.sliit.snapslip`, version 1.0 (1)	Done
Generated launch screen; iPhone and iPad orientations	Done
Camera usage description explaining on-device processing	Done
Shared scheme with tests and code coverage	Done

Live demonstration plan (about 6 minutes)

1. **Launch** (30 s): show the icon, then onboarding with name and budget.
2. **Scan** (90 s): open the scanner, choose the supermarket sample, point out the light sweep, the text boxes, the suggested category, its confidence and its engine. Save.
3. **Learn** (60 s): scan a receipt from a new shop, change its category, scan it again and show the change was remembered; open *Learned shops*.
4. **Explore** (60 s): Overview ring and projection; Insights donut selection; search and filter in Expenses; pinch-zoom a receipt.
5. **Adapt** (45 s): switch to Dark Mode and the largest text size; rotate to landscape or show iPad.
6. **Quality** (45 s): run a few unit tests in Xcode and show the test report.
7. **Hand-off to Part B** (30 s): export the CSV for SnapSlip Spatial.

Backup: if the camera or OCR misbehaves during the demo, launch with `-uiTesting -seedSampleData` to show fully populated screens.

Part B -SnapSlip Spatial for Apple Vision Pro

11. SnapSlip Spatial: concept and prototype

SnapSlip Spatial turns a month of receipts into a space you can stand in. Spending becomes something you see at room scale, not another pie chart. It is the Vision Pro companion to the iPhone app and reads the same CSV export.

11.1 Concept

- **Spending City (volumetric window).** Each category is a tower on a 3×3 plot, and its height is that month's spend. Pinch a tower to select it and see its amount; drag to spin the city; step months in the ornament and watch the towers grow and shrink.
- **Budget Orbit (immersive space, mixed).** Each category becomes a glowing orb on an arc 1.5 m in front of you, sized by spend (area proportional to amount). A ring of 60 beads on the floor around you lights up as the budget is used and turns red when it is exceeded. Pinch an orb to open its receipts beside it, drag orbs to rearrange them, and magnify one with both hands.
- **Dashboard (window).** Month sidebar, budget gauge, category cards with hover effects, receipt list, and import from iPhone.

The unique need: people understand budgets poorly from numbers alone. Seeing that "dining" is the tallest tower, or that the ring around you is two-thirds lit by the 10th of the month, makes overspending physical and memorable. This makes it useful for personal reflection and for financial-literacy teaching.

11.2 Similar apps

App	What it does on Vision Pro	Gap SnapSlip Spatial fills
NatWest banking app	Retail banking in 3D windows from August 2024: payments, transfers and Direct Debits, controlled by eyes, hands and voice	Transactions in flat windows; no spatial view of spending habits
MoneyCoach	Budgeting and goals across iPhone, iPad, Mac and Watch; lists visionOS 2 support; Premium $9.99/month or $29.99/year	A conventional 2D budgeting interface in a window
Tableau for Vision Pro	Research app with 3D bar and line charts and data globes, explored with gaze and gestures; beta-tested with 22 people at Tableau Conference 2024	Business analytics for analysts; not personal, not built from receipts
Swift Charts 3D	Apple framework (WWDC25) for 3D scatter and surface charts on iOS, macOS and visionOS 26	A building block, not an app; a possible upgrade for Insights

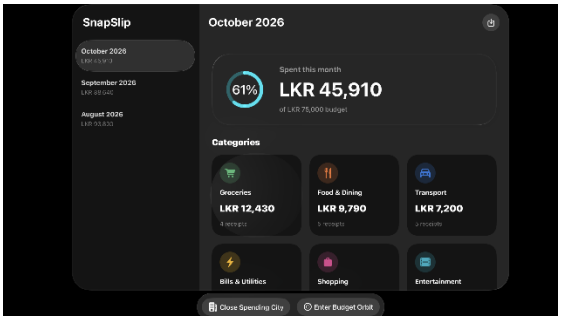
No existing app turns a person's own captured receipts into a space they can stand inside.

11.3 Prototype functionality

The prototype implements all three visionOS scene types and keeps them in sync through one shared model.

Scene	Platform capability	What works
Dashboard window	Glass window with a sidebar, toolbar and bottom ornament	Month list with totals; budget gauge; category cards; receipt list for the selected category; import a CSV from the iPhone app; open and close the other two scenes
Spending City	Volumetric window (0.9 × 0.6 × 0.9 m)	Nine towers sized by spend that animate when the month changes; pinch a tower to select it; drag to spin; floating labels with amounts; month stepper ornament
Budget Orbit	Immersive space, mixed immersion	Category orbs on an arc 1.5 m away, sized by spend and gently floating; 60-bead floor ring lit by budget used, red when over; pinch an orb to open its receipt card; drag to move; magnify to resize; summary panel with month stepper

Selecting a category in any scene highlights it in the others, and changing the month in the volume or the immersive space updates the window as well.





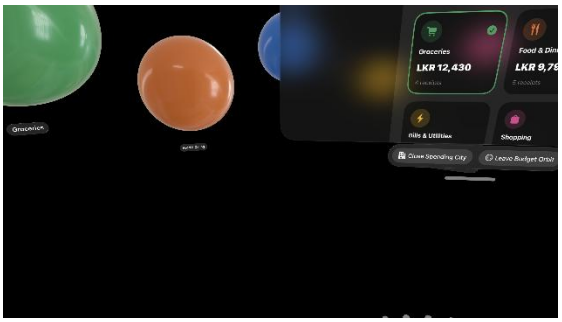






Figure 11.1 -The three-layer visionOS app icon (back, middle and front layers shown combined).

11.4 Interaction and user experience design

Every interaction uses the standard visionOS input model -look, then pinch -so nothing has to be learned.

User action	Result	Platform convention followed
Look at a card, tower or orb	It highlights	hoverEffect and HoverEffectComponent give gaze feedback before any tap
Look and pinch	Selects the category; opens its receipts	SpatialTapGesture targeted to entities
Pinch and drag a tower	Spins the city	Direct manipulation of a volume's content

User action	Result	Platform convention followed
Pinch and drag an orb	Moves it; floating pauses while held	Objects stay where the user puts them
Two-hand magnify an orb	Resizes it between set limits	Standard magnify gesture with clamped scale
Ornament buttons	Open or close the volume and immersive space; step months	Controls attached to the window, not floating in space

Design guideline choices:

- **Comfort.** Content sits 1.1–1.5 m from the user in mixed immersion, so the room stays visible. Nothing moves the user's viewpoint, which avoids motion discomfort.
- **Legibility.** Labels and cards are SwiftUI views on glass, attached to entities with `BillboardComponent` so they always face the user.
- **Progressive depth.** The user starts in a familiar window, opens a small volume, and only enters the immersive space by choice.
- **Accessibility.** Each tower and orb has an `AccessibilityComponent` with the category name and amount, so VoiceOver can read the 3D scene. Colour is backed by labels and the over-budget message.

11.5 Technical implementation

Capability	Implementation
Multiple scene types	`WindowGroup` (dashboard), `WindowGroup` + `.windowStyle(.volumetric)` (city), `ImmersiveSpace` with mixed immersion (orbit)
Shared state	One `@Observable AppModel` injected into all three scenes, so selection and month stay in sync
Entity-Component-System	`CategoryComponent` (tags entities for gestures), `TowerComponent` (target height, avoids restarting animations), `FloatComponent` with a custom `FloatSystem` that bobs orbs every frame and pauses while dragged
Procedural 3D	`MeshResource` boxes and spheres; `PhysicallyBasedMaterial` with clearcoat and emission for the orbs; no USDZ assets
Spatial UI	`RealityView` attachments for labels, receipt cards and the summary, with `BillboardComponent` so they face the user; ornaments for controls; `glassBackgroundEffect`
Input	`SpatialTapGesture`, `DragGesture` and `MagnifyGesture`, all `.targetedToAnyEntity()`; `InputTargetComponent`, `CollisionComponent` and `HoverEffectComponent` on every interactive entity

Capability	Implementation
Accessibility	`AccessibilityComponent` label and value on each tower and orb
Animation	`Entity.move(to:relativeTo:duration:timingFunction:)` for tower heights and orb sizes when the month changes
Data	Sample data and a CSV importer compatible with the iOS export (quoted fields, CRLF)
Testing	Swift Testing for CSV import, layout maths (`SpatialLayout`) and `AppModel`

Comfort follows Apple's design guidelines: content sits 1.1–1.5 m away, immersion is mixed so the room stays visible, nothing moves the user, and every label is billboarded and on glass.

11.6 Performance and stability

- **Build once, update in place.** Towers, orbs and the 60 beads are created once. Month changes only adjust scale, position and materials; nothing is rebuilt.
- **Animate only on change.** `TowerComponent` stores each tower's target height, so a redraw that doesn't change the data starts no animation.
- **Cheap per-frame work.** The custom `FloatSystem` runs one sine calculation per orb per frame and skips orbs being dragged.
- **Stable attachments.** The receipt card is a permanent attachment that is shown or hidden with `isEnabled`, instead of being created and destroyed on every selection.
- **Safe scene transitions.** An open/closing/closed state machine stops the immersive space being opened twice while a transition is running.
- **Tolerant import.** The CSV importer handles quoted fields, Windows line endings and bad rows without crashing, and reports a clear error for the wrong file type.

11.7 Pitch summary

The investor pitch is a separate 12-slide deck, SnapSlip Spatial -Investor Pitch. Its argument in brief:

Pitch element	Summary
Value proposition	iPhone removes the typing from budgeting; Vision Pro makes the month something you can see and stand inside. Private, on-device, and personal.
Target market	Beachhead: Sri Lankan university students and young professionals on iPhone. Then Vision Pro early adopters, banks and financial-literacy educators. Personal finance software was about USD 1.4 billion in 2025, forecast to reach USD 2.1 billion by 2034 (IMARC). Vision Pro remains niche: about 390,000 units in 2024 and an IDC estimate of about 45,000 in Q4 2025 (The Register), which is why the business starts on iPhone.
Revenue model	Freemium iPhone app with a monthly scan allowance; SnapSlip Plus subscription that includes unlimited scans and the Vision Pro app; per-seat licences for banks (white-label insights) and schools (financial-

Pitch element	Summary
	literacy sessions).
Go-to-market	Months 0–3: launch on one campus with student ambassadors and budgeting challenges. Months 3–9: release on the visionOS App Store and launch Plus. Months 9–18: pilot with one bank and one education partner.
Impact	Better money habits for young people through capture that takes seconds and a view of spending that is hard to ignore.

Supporting material

12. User guide

This guide is for people using the apps. Build and run instructions for developers are in each project's README.

SnapSlip on iPhone and iPad

Getting started

1. Open SnapSlip and swipe through the three intro pages.
2. On the last page, enter your name, pick your currency and set a monthly budget. Tap **Get started**.
3. Optional: go to **Settings → Load sample data** to explore with four months of example spending.

Scan a receipt

1. Tap the round scan button at the bottom right of Overview or Expenses. On iPad, use **Scan receipt** in the sidebar.
2. Choose **Scan with camera** (hold the phone over the receipt; it captures automatically), **Choose from Photos**, **Try a sample receipt**, or **Enter manually**.
3. Wait a moment while SnapSlip reads the receipt. On the review screen, check the shop, amount and date. Tap the text icon on the image to show or hide what was read.
4. Check the suggested category. The bar shows how confident SnapSlip is and which engine decided. Tap a different category chip if it's wrong; SnapSlip will remember your choice for that shop.
5. Tap **Save expense**. The new expense opens in Expenses.

Add an expense manually: tap + on Overview or Expenses, fill in the shop and amount, choose a category, and tap **Save**.

Find and manage expenses (Expenses tab)

- Type in the search bar to find a shop or note. Tap a category chip to filter.
- Swipe a row left to delete or right to edit. Touch and hold for a preview, duplicate or delete.
- Tap a row for full details. Tap the receipt to view it full screen: pinch to zoom, double-tap to zoom in or out, drag down to close.

Understand your spending

- **Overview:** the ring shows how much of your monthly budget is used. It turns orange at 85% and red when you go over. Below it are today's spend, your daily average and the projected month-end total.
- **Insights:** choose This month, 3 months or 6 months. Touch and drag around the ring chart to explore categories. The pace chart compares your running total with an even daily budget.
- Tap any category to see its expenses and its budget.

Settings

Setting	What it does
Monthly budget, Currency	Your overall limit and the currency for new expenses
Category budgets	Optional limits per category, shown as progress bars
Theme, Mode	Accent colour (Lagoon, Sunset, Tea Estate, Orchid) and Light, Dark or System
Learn from my corrections, Learned shops	Turn personal learning on or off; review, change or delete what was learned
Export as CSV	Share all expenses to Files, Mail or AirDrop, or to SnapSlip Spatial
Erase all data	Permanently deletes expenses, receipts, budgets and learned shops

SnapSlip Spatial on Apple Vision Pro

1. Open SnapSlip Spatial. The dashboard shows months on the left and categories on the right. Look at a category card and pinch to see its receipts.
2. To use your own data, tap **Import from iPhone** and choose the CSV exported from SnapSlip.
3. Tap **Open Spending City** in the bar under the window. A small city appears. Look at a tower and pinch to select it. Pinch and drag to spin the city. Use the arrows below it to change month.
4. Tap **Enter Budget Orbit**. Orbs appear in front of you and a ring of beads on the floor shows your budget used. Pinch an orb to open its receipts. Pinch and drag to move an orb, or use both hands to resize it.
5. Tap **Leave Budget Orbit** to return to your room.

13. Reflections

What worked well. Treating machine learning as layers rather than a single model was the most important decision. The app was useful before the Core ML model existed, because keyword rules and embeddings covered the gap, and the personal layer added something a generic model cannot: it adapts to each user. Keeping the parser, analytics and CSV code free of UI and framework dependencies made them easy to unit-test, and writing those tests exposed real mistakes, such as "Ground Coffee" being dropped as a non-item because it contains "round".

What was harder than expected. Real receipts are messy. Vision splits rows, prints totals on the next line and mixes subtotals, cash and change, so most of the effort went into the parser rather than the OCR call itself. Coordinate spaces were a second trap: Vision's bottom-left, normalised boxes had to be flipped for SwiftUI. SwiftData looked simple but needed care with relationships, to avoid orphaned line items and to avoid showing an object that had just been deleted. On visionOS, the challenge was designing for gaze and pinch, keeping content at a comfortable distance and making labels readable in 3D.

What testing taught me. [Summarise the most useful findings from the automated tests and the user acceptance test, and what you changed as a result.]

What I would do next.

- Sinhala and Tamil localisation with a String Catalog, including receipts printed in those scripts.
- App Intents and Siri ("How much have I spent on dining this month?") and a home-screen widget for the budget ring.
- iCloud sync with CloudKit, and shared household budgets explored together in SharePlay on Vision Pro.
- Swift Charts 3D in the visionOS app, and multi-page receipts in the scanner.

What I learned. Building for two platforms showed how much of an app is shared logic and how much is presentation. The same spending data became a ring on iPhone and a city in space, and keeping the model, services and views separate is what made that possible. I also learned to treat AI-generated code as a draft that has to be built, tested and understood line by line before it can be trusted (section 14).

14. References

1. Apple Inc. (2025) *Bring Swift Charts to the third dimension*, WWDC25 session 313. Available at: developer.apple.com/videos/play/wwdc2025/313.
2. Apple Inc. (n.d.) *Machine Learning* -Core ML, Create ML, Vision and Natural Language. Available at: developer.apple.com/machine-learning.
3. Apple Inc. (n.d.) *Human Interface Guidelines: Designing for visionOS*. Available at: developer.apple.com/design/human-interface-guidelines/designing-for-visionos.
4. Apple Inc. (n.d.) *SwiftData*, *RealityKit*, *Swift Charts* and *Swift Testing* documentation. Available at: developer.apple.com/documentation.
5. IMARC Group (2025) *Personal Finance Software Market*. Available at: imarcgroup.com.
6. NatWest Group (2024) *Launching our retail banking app on Apple's Vision Pro*, 1 August. Available at: natwestgroup.com.
7. 9to5Mac (2024) *Indie App Spotlight: MoneyCoach*, 28 September. Available at: 9to5mac.com.
8. Tableau (2024) *Exploring spatial computing and immersive analytics on Apple Vision Pro*. Available at: tableau.com.
9. The Register (2026) *Headset hype meets harsh reality as Apple and Meta VR shipments fizzle in 2025*, 2 January. Available at: theregister.com.
10. Slashdot (2026) *IDC estimates Apple shipped just 45,000 Vision Pros last quarter*, 1 January. Available at: apple.slashdot.org.

Appendix A -Build and run

Step	SnapSlip (iOS)	SnapSlip Spatial (visionOS)
Requirements	Xcode 16 or later; iOS 17+ simulator or device	Xcode 16 or later with the visionOS platform; visionOS 2+ simulator
Open	SnapSlip.xcodeproj	SnapSlipSpatial.xcodeproj
Sign	Select your team for the SnapSlip target	Select your team for the SnapSlipSpatial target
Run	⌘R on an iPhone simulator; use *Try a sample receipt*	⌘R on the Apple Vision Pro simulator
Test	⌘U (unit and UI tests, coverage on)	⌘U
Optional	cd MLTraining && swift TrainClassifier.swift, then add the .mlmodel to the app	Import a CSV exported from SnapSlip
Fallback	xcodegen generate using project.yml	—

Appendix B -Project structure

```
SnapSlip/
├── SnapSlip/
│   ├── App/           entry point, router, services container, settings keys
│   ├── Models/        SwiftData models, ExpenseDraft, categories
│   ├── Persistence/   container and schema, repository, merchant memory, sample data
│   ├── Services/      Vision OCR, parser, categoriser, images, analytics, CSV
│   ├── DesignSystem/  theme, modifiers, button styles, custom components
│   ├── Views/         Root, Onboarding, Dashboard, Expenses, Scan, Insights, Settings
│   └── Resources/     asset catalogue (icons, colours)
├── SnapSlipTests/     Swift Testing unit tests (5 suites)
├── SnapSlipUITests/   XCUITest UI and performance tests
├── MLTraining/        receipt_categories.csv, TrainClassifier.swift
├── README.md, AI_USAGE.md, project.yml

SnapSlipSpatial/
├── SnapSlipSpatial/
│   ├── App/           three scenes, shared AppModel
│   ├── Model/         SpendingStore, CSVImporter
│   ├── RealityKit/    components and FloatSystem, EntityFactory, SpatialLayout
│   ├── Views/         DashboardWindow, SpendingCityView, BudgetOrbitView
│   └── Resources/     layered app icon, accent colour
├── SnapSlipSpatialTests/
├── Config/Info.plist  scene manifest
├── README.md, AI_USAGE.md
```

Appendix C -Test inventory

Target	Suite	Test functions	Framework
SnapSlipTests	Receipt parser	13	Swift Testing
SnapSlipTests	Expense categoriser	7	Swift Testing
SnapSlipTests	Spending analytics	8	Swift Testing
SnapSlipTests	Persistence	9	Swift Testing
SnapSlipTests	Scan pipeline and helpers	9	Swift Testing
SnapSlipUITests	UI flows	8	XCUITest
SnapSlipUITests	Performance	2	XCTest metrics
SnapSlipSpatialTests	CSV import, spatial layout, app model	12	Swift Testing
Total		**68**	

Parameterised tests run once per argument, so the number of executed test cases is higher than the number of functions.